

AkomaPlanner — Code Map

Where to find each feature's code, and what it does.

Companion to README.md (how to run) and LEARNING.md (why it's built this way).

How this app's folders are organized

<code>src/app/</code>	Pages and routes (Next.js App Router — one folder per URL)
<code>src/components/</code>	UI building blocks, organized by feature folder
<code>src/lib/</code>	Business logic + data access (the "service layer") — no UI code here
<code>src/server/actions/</code>	"use server" functions that call lib/ and touch the database
<code>prisma/</code>	schema.prisma (data model) and seed.ts (sample data)

Landing Page (public, logged-out view)

`/`

`src/app/page.tsx`

The marketing page shown before you're inside the app — hero, feature cards, and the AI-safety commitments section. Both call-to-action buttons now point to `/signup` instead of straight into onboarding.

Account Creation & Login [new]

`/signup`, `/login`, `/forgot-password`, `/verify-email`

`src/app/signup/page.tsx` + `src/components/auth/SignUpForm.tsx`

Real account creation — name, email, password, confirm-password. On success, signs you in and sends you straight into the onboarding wizard.

`src/app/login/page.tsx` + `src/components/auth/LogInForm.tsx`

Log back in with an email/password created via sign-up (or the seeded demo account).

`src/components/auth/SocialLoginButtons.tsx`

Google/Apple/Facebook/Microsoft buttons — visibly present but genuinely disabled ('Soon' badge), not silently fake. No OAuth is wired up yet.

`src/app/forgot-password/page.tsx`

Honest placeholder — shows a static confirmation, no email is actually sent, no backend call at all.

`src/app/verify-email/page.tsx`

Honest placeholder — no verification email is sent. A clearly-labelled demo button lets you simulate clicking one.

`src/server/actions/auth.ts`

`signUp()` / `login()` / `signOut()` / `verifyEmailDemo()` — the four real account actions.

`src/lib/auth.ts`

`hashPassword()` / `verifyPassword()` (real password hashing via Node's built-in `crypto.scrypt`), plus the session-cookie functions. See LEARNING.md #34 for what's real vs. still a placeholder here.

Onboarding (9-step wedding setup wizard) [updated]

`/onboarding`

`src/app/onboarding/page.tsx`

Checks whether you already have a wedding plan; redirects to the dashboard if so, otherwise shows the wizard.

`src/components/onboarding/OnboardingWizard.tsx`

Holds all wizard state in one object across all 9 steps, shows a progress bar, and submits once on the final step.

src/components/onboarding/steps/*.tsx

One component per step: CoupleInfoStep, WeddingDetailsStep, GuestInfoStep, BudgetStep, StyleStep, PreferencesStep, VendorStatusStep, RelationshipStep, NotesStep.

src/lib/validation/wedding.ts

onboardingSchema — the zod validation rules for every wizard field, plus the

ONBOARDING_VENDOR_CATEGORIES list the Vendor Status step and dashboard card both read from.

src/server/actions/wedding.ts

createWeddingPlan() — validates the whole wizard's answers, then creates the wedding, its CoupleProfile, its VendorBookingStatus rows, its default checklist, and starter budget, all in one go.

Dashboard [updated]

/dashboard

src/app/(app)/dashboard/page.tsx

The home screen: personalized greeting, countdown, budget snapshot with an animated progress ring, checklist progress ring, this week's high-priority focus, vendor-booking status, wedding-style card, recommended accommodation, quick links.

src/components/dashboard/StatCard.tsx

The small labelled-number tile used across the dashboard.

src/components/dashboard/ProgressRing.tsx

The circular 'meter' used for Budget-spent and Checklist-progress — one hue for the fill, the same hue faint for the track.

src/components/dashboard/VendorStatusCard.tsx [new]

Shows the 12-category vendor-booking checklist collected during onboarding.

src/components/dashboard/WeddingStyleCard.tsx [new]

Displays the couple's theme/dress-code/color-palette as text + swatches — informational only, doesn't re-skin the app.

Budget [updated]

/budget

src/app/(app)/budget/page.tsx

Add-category form up top, summary stat cards, an animated total-spent progress bar, then the full editable category table.

src/components/budget/BudgetCategoryRow.tsx

One editable row — allocated/spent inputs save automatically.

src/components/budget/AddBudgetCategoryForm.tsx

Form for adding a new budget category.

src/components/budget/BudgetProgressBar.tsx [new]

The animated horizontal 'meter' showing total spent vs. total budget.

src/server/actions/budget.ts

add/update/delete a budget category.

src/lib/budget.ts

The actual math (totals, percentages, over-budget flags) — pure functions, unit-tested in tests/budget.test.ts.

src/lib/budget-defaults.ts

The starter category split (Venue 25%, Catering 20%, etc.) used when a wedding is first created.

Checklist [updated]

[/checklist](#)

src/app/(app)/checklist/page.tsx

Add-task form up top, progress stats, then tasks grouped by category.

src/components/checklist/ChecklistItemRow.tsx

One checkbox row — toggle done / change priority (Low/Medium/High) / remove.

src/components/checklist/AddChecklistItemForm.tsx

Form for adding a custom task, including its priority (capped at 15 on Free).

src/server/actions/checklist.ts

toggle/add/delete a checklist item, and updateChecklistItemPriority().

src/lib/checklist-defaults.ts

The default Ghana-relevant checklist template (with a priority pre-assigned to every default task) and the function that turns it into real due dates.

Guest List [updated]

[/guests](#)

src/app/(app)/guests/page.tsx

Stat cards + RSVP pie chart, an add-guest form, then the guest table.

src/components/guests/GuestRow.tsx

One guest row — RSVP dropdown + remove.

src/components/guests/AddGuestForm.tsx

Add-a-guest form (capped at 100 on Free).

src/components/guests/RsvpPieChart.tsx [new]

Donut chart of confirmed/pending/declined guests, with a legend.

src/server/actions/guests.ts

add guest / update RSVP / delete guest.

Vendors

[/vendors](#)

src/app/(app)/vendors/page.tsx

Browse/filter vendor listings by category and city.

src/app/(app)/vendors/[vendorId]/page.tsx

One vendor's detail page + the enquiry/negotiation panel.

src/app/(app)/vendors/compare/page.tsx

Pro side-by-side comparison table for selected vendors.

src/components/vendors/VendorCard.tsx

One vendor listing card (locked/blurred if Pro-featured and you're on Free).

src/components/vendors/VendorFilters.tsx

Category/city filter dropdowns.

src/components/vendors/VendorGrid.tsx

Wraps the vendor cards, manages which ones are checked for comparison.

src/components/vendors/VendorInterestPanel.tsx

The whole draft -> send -> quote -> approve negotiation flow for one vendor.

src/server/actions/vendors.ts

Every step of the negotiation lifecycle — the ONLY code allowed to change a vendor enquiry's status.

BisaAI (chat + AI tools) [updated]

[/bisaai](#), and a floating bubble on every app page

src/app/(app)/bisaai/page.tsx

Free Q&A chat plus six Pro tool cards, plus links out to tools with their own screens.

src/components/bisaai/ChatPanel.tsx

The free Q&A chat box on the full [/bisaai](#) page.

src/components/bisaai/FloatingChatBubble.tsx [new]

A collapsible chat bubble mounted once in `(app)/layout.tsx` — available on every app page, not just [/bisaai](#).

src/components/bisaai/ShoppingListTool.tsx, DecorMoodboardTool.tsx, HoneymoonTool.tsx,

The six inline Pro tool cards.

src/server/actions/bisaai.ts

Thin wrappers that add Free/Pro gating in front of `lib/bisaai.ts`.

src/lib/bisaai.ts

THE central AI module — all 13 mock BisaAI functions live here. Start here to swap in a real AI provider.

Traditional List Pricing

[/traditional-list](#)

src/app/(app)/traditional-list/page.tsx

Pro screen showing estimated drinks/kente/gifts pricing by market, grouped by category.

Calendar

[/calendar](#)

src/app/(app)/calendar/page.tsx

Month view — wedding day, checklist due dates, and (Pro) vendor activity.

src/lib/dates.ts

Date helper functions, including the month-grid math the calendar is built on.

Accommodation [now free]

[/accommodation](#)

src/app/(app)/accommodation/page.tsx

Free for every plan now — lodging near the venue, sorted by distance, filterable by city. A richer Pro version is a deferred idea, not built.

Collaboration + Public Share Page

[/collaboration](#) and [/w/\[shareToken\]](#)

src/app/(app)/collaboration/page.tsx

Pro: manage who else can see this wedding, plus the public share link.

src/components/collaboration/MemberList.tsx

List of collaborators + the add-collaborator form.

src/components/collaboration/ShareLinkPanel.tsx

Copy/regenerate the public share link.

src/server/actions/collaboration.ts

add/remove collaborator, regenerate share token.

src/app/w/[shareToken]/page.tsx

The public, no-login page anyone with the link can see (names/date/countdown only — no budget or guest details).

Design Tools

[/design](#)

src/app/(app)/design/page.tsx

Pro: wedding-day timeline, invitation template gallery, and seating chart.

src/components/design/InvitationTemplates.tsx

Three real (non-AI) invitation layout templates.

Dress Try-On (experimental)

[/dress-tryon](#)

src/app/(app)/dress-tryon/page.tsx

Pro: the consent-gated experimental preview screen.

src/components/dress-tryon/DressTryOnForm.tsx

Upload UI + consent checkbox + placeholder result — your photo never leaves the browser.

Profile [new]

[/profile](#)

src/app/(app)/profile/page.tsx + src/components/profile/ProfileForm.tsx

Editable view of everything the onboarding wizard collected — fill in more or change anything, any time.

src/server/actions/profile.ts

updateCoupleProfile() — validates and saves changes.

src/lib/validation/profile.ts

Reuses onboardingSchema.pick() so the two forms' validation rules can't drift apart.

Settings + Export [updated]

[/settings](#)

src/app/(app)/settings/page.tsx

Account info (with a Sign Out button and a Verify Email link), current plan, a link to /profile, and the two PDF export buttons.

src/app/api/export/schedule-pdf/route.ts

Free: downloads your checklist as a PDF.

src/app/api/export/full-report-pdf/route.ts

Pro: checklist + budget + guest summary as one PDF.

src/lib/pdf.ts

The actual PDF-drawing code both export routes use.

Pricing + Checkout

[/pricing](#) and [/checkout](#)

src/app/pricing/page.tsx

Free vs. Pro comparison table with the billing-interval toggle.

src/components/pricing/PricingCards.tsx

The toggle + both plan cards.

src/app/checkout/page.tsx

Stubbed checkout — one confirm button, no payment fields.

src/server/actions/billing.ts

upgradeToPro() / downgradeToFree() — flips the plan; this is the file to edit for real Paystack/Hubtel integration.

src/lib/pricing.ts

The GHS 62/500/188 price constants, shared by the pricing and checkout pages.

Behind every page: the shared foundations

Layout & navigation

src/app/layout.tsx

Root layout — wraps every single page, loads global CSS.

src/app/(app)/layout.tsx

Wraps every 'inside the app' page with the sidebar + top bar + the floating BisaAI chat bubble; redirects to onboarding if you have no wedding yet.

src/components/layout/Sidebar.tsx

The left nav — every feature is listed here, Pro ones with a badge, plus Pricing/Profile/Settings.

src/components/layout/TopBar.tsx

Couple name, countdown, plan badge shown above every app page.

src/components/layout/PublicHeader.tsx [new]

The logo + back-link header shared by /signup, /login, /forgot-password, /verify-email.

Feature gating (Free vs. Pro)

src/lib/plan.ts

requirePro() and the Free-tier limit checks (1 wedding, 100 guests, 15 custom tasks) — the ONE place gating rules live.

src/components/pro/UpgradePrompt.tsx

The standard 'this is a Pro feature' block shown wherever a gate blocks you.

Data & accounts [updated]

prisma/schema.prisma

The entire data model — every table, every field, with explanatory comments. Now includes password/auth fields on User, plus new CoupleProfile and VendorBookingStatus tables for onboarding data.

prisma/seed.ts

Generates all the sample data: 38 vendors, traditional-list items, accommodations, one fully-populated sample wedding (with a real login password too).

src/lib/prisma.ts

The shared database connection used everywhere.

src/lib/session.ts

getCurrentUser() / getCurrentWeddingPlan() — checks the real session cookie first, falls back to the original seeded demo account if you're not signed in.

src/lib/auth.ts [new]

Password hashing (crypto.scrypt) and the session-cookie functions — see the Account Creation section above.

src/lib/validation/wedding.ts

The zod validation rules (onboardingSchema) for the whole onboarding wizard.

src/lib/validation/auth.ts [new]

The zod validation rules for sign-up and log-in.

src/lib/validation/profile.ts [new]

The zod validation rules for the /profile edit form (reuses onboardingSchema.pick()).

src/lib/form-shaping.ts [new]

Small shared helpers (orUndefined, dateOrUndefined) used by both the onboarding and profile server actions.

Small shared helpers [updated]

src/lib/currency.ts

formatGHS() — consistent GHS money formatting everywhere, whole cedis only (no decimals).

src/components/ui/Button.tsx, Card.tsx, Badge.tsx

The basic visual building blocks almost every screen is made from.

src/components/ui/Input.tsx, Select.tsx, Textarea.tsx, Field.tsx [new]

Shared form-field primitives, introduced for the onboarding wizard and profile form (~45 fields made copy-pasted styling worth collapsing).

Tip: every file in this app starts with a header comment explaining what it does and why — this map tells you WHERE to look; the file itself tells you WHY it's built that way.

Languages & Technologies Used Here

A study guide, not just a list — what each one is, where it shows up in this codebase, and where to go read more if you want to go deeper.

TypeScript

THE PRIMARY LANGUAGE — [start here](#)

Where: Everywhere. Every .ts and .tsx file in src/, prisma/seed.ts, this app's tests.

What it's for: JavaScript with types added — catches whole categories of bugs (wrong field names, null values, mismatched shapes) before the code ever runs. If you only study one thing from this list, study this.

Learn more: typescriptlang.org/docs (the official Handbook)

React (as JSX/TSX syntax)

The UI library

Where: Every file under src/components/ and every page under src/app/*/page.tsx.

What it's for: Building the actual screens out of reusable components. Look at useState/useTransition in a form component (e.g. src/components/guests/AddGuestForm.tsx) to see the core pattern used everywhere in this app.

Learn more: react.dev/learn

Next.js (App Router)

The framework wrapping React

Where: src/app/'s folder structure, every 'use server' file under src/server/actions/, every 'use client' component.

What it's for: File-based routing (a folder = a URL), Server Components (render on the server, default), Client Components ('use client', for interactivity), and Server Actions ('use server', for form submissions without hand-written API endpoints).

Learn more: nextjs.org/docs

Tailwind CSS

The styling approach

Where: The className on nearly every element in every component.

What it's for: Utility classes (bg-akoma-green, rounded-lg, px-4) instead of separate .css files — the brand color tokens themselves are defined once in tailwind.config.ts.

Learn more: tailwindcss.com/docs

Prisma (schema language + client)

The database layer

Where: prisma/schema.prisma (the data model itself), and prisma.<model>.findMany/create/update calls throughout src/lib/ and src/server/actions/.

What it's for: Prisma's schema.prisma file is its own small, readable language for describing database tables and relationships; the Prisma Client it generates gives fully-typed database queries in TypeScript, so a typo in a field name is caught before the app even runs.

Learn more: prisma.io/docs

SQL / relational databases

What Prisma sits on top of

Where: prisma/dev.db (SQLite, a real single-file SQL database) — every prisma.* call becomes real SQL underneath.

What it's for: Worth understanding independently of Prisma: tables, rows, foreign keys (e.g. WeddingPlan.ownerUserId points at a User), and one-to-one/one-to-many relationships (a WeddingPlan has one CoupleProfile but many Guests).

Learn more: sqlitetutorial.net, or any general relational-database/SQL primer

Zod

Input validation

Where: Every file under src/lib/validation/, imported by nearly every src/server/actions/ file.

What it's for: Describes 'what a valid form submission looks like' once, then both validates real input server-side (the actual security boundary) and gives TypeScript the exact type of validated data — one schema, two jobs.

Learn more: zod.dev

Node.js built-ins

The JavaScript runtime underneath

Where: `src/lib/auth.ts` imports `crypto` directly (scrypt for password hashing) — no external library needed for that.

What it's for: Worth knowing Node ships a real standard library — cryptography, file system, etc. — without installing anything.

Learn more: nodejs.org/docs (see the 'crypto' module)

Markdown

Documentation, not app code

Where: `README.md`, `LEARNING.md`, and this document's source content.

What it's for: Plain-text formatting for docs — headings, lists, code blocks. Every serious software project has files like these.

Not used in this project

No Python, Java, Go, Rust, PHP, or Ruby anywhere in this codebase — it's a single-language (TypeScript) full-stack app, front end and back end both. That's a deliberate, common modern choice: one language for the whole team/project to know, one type system shared between the browser code and the server code.